

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Case Study

COURSE NAME: COMPUTER VISION

COURSE CODE:ITA0515

Slot B

COURSE OUTCOME COVERED

CO3: *Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BTL 4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks 50

Duration : 60 Minutes

Annexure A

Case Study

Code of Conduct:

I, S. Durga Prasad (192472091) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Prasad

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks(100)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly	
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts	
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning	
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided	
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand	

1. Preprocessing and Feature Detection

A vision system processes images captured under varying lighting conditions. Without preprocessing, feature detection results are inconsistent . Analyze the issue and explain how preprocessing improves feature detection performance.

Answer :

In a computer vision system, images may be captured under **different lighting conditions**, such as bright light, low light, shadows, or uneven illumination. These changes can affect the intensity, contrast, and visibility of important features in an image. As a result, feature detection algorithms may produce inconsistent or incorrect results.

Problems caused by varying lighting conditions

1. **Change in brightness:** The same object may appear brighter or darker depending on the illumination.
2. **Low contrast:** Important edges and features may become difficult to detect.
3. **Shadows and highlights:** Shadows can create false edges, while bright regions may hide actual features.
4. **Noise:** Low-light images often contain more noise, which can be detected as unwanted features.
5. **Inconsistent feature points:** Algorithms such as SIFT, SURF, ORB, or corner detectors may detect different points in images of the same object.

Role of preprocessing

Preprocessing improves the image before feature detection. Common preprocessing techniques include:

1. Grayscale conversion

Converts a color image into a single intensity channel, reducing computational complexity while retaining important structural information.

2. Noise reduction

Filters such as **Gaussian Blur** or **Median Blur** remove unwanted noise. This prevents noise from being incorrectly detected as features.

3. Contrast enhancement

Techniques such as **Histogram Equalization** and **CLAHE** improve the visibility of features in dark or low-contrast images.

4. Illumination normalization

Corrects uneven lighting so that the same object has more consistent intensity values under different lighting conditions.

5. Image normalization

Pixel intensities can be scaled to a common range, such as 0–1 or 0–255, making images more consistent for further processing.

6. Sharpening

Sharpening techniques can enhance important edges and fine details, making them easier for feature detectors to identify.

Example workflow

Input Image → Grayscale → Noise Reduction → Contrast Enhancement → Illumination Normalization → Feature Detection

For example, if a face is photographed once in bright light and once in a dark room, preprocessing can reduce the difference between the two images. The feature detector can then identify similar **edges, corners, and keypoints** more reliably.

Advantages

- Improves feature detection accuracy.

- Reduces false and unwanted features.
- Makes images more consistent under different lighting conditions.
- Improves edge and corner detection.
- Reduces the effect of image noise.
- Makes subsequent tasks such as **object recognition, image matching, tracking, and classification** more reliable.

Conclusion

Preprocessing is an important stage in a computer vision pipeline because **feature detection depends heavily on image quality and illumination**. Techniques such as grayscale conversion, filtering, contrast enhancement, normalization, and illumination correction reduce unwanted variations and enhance useful information. Therefore, preprocessing produces **more stable, accurate, and repeatable feature detection results** even when images are captured under different lighting conditions.

2. Edge vs Corner Detection

In an image containing both straight edges and textured regions, a system uses edge detection but fails to identify key points for matching. Analyze why edge detection is insufficient and justify the use of corner detection in this case.

Answer :

In computer vision, **edge detection** and **corner detection** are used to identify important structures in an image. If an image contains straight edges as well as textured regions, using only edge detection may not provide enough information for **image matching**.

Why Edge Detection is Insufficient

An **edge** is a region where the intensity of an image changes significantly, usually representing boundaries between different objects or regions.

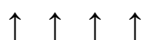
For example, a straight line such as the edge of a table can be detected easily using methods like **Canny** or **Sobel**.

However, edges have some limitations:

1. **Edges are not unique:** A long straight edge may contain many points with similar information.
2. **Poor localization for matching:** It is difficult to determine which particular point on a straight edge corresponds to a point in another image.
3. **Lack of two-dimensional information:** Along a straight edge, intensity changes mainly in one direction, so there is less information for identifying a unique location.
4. **Sensitive to noise and lighting:** Small changes in image conditions can create unwanted edges or remove weak edges.
5. **Difficult feature matching:** Edge detection produces lines or boundaries rather than distinctive keypoints.

For example:

Straight Edge



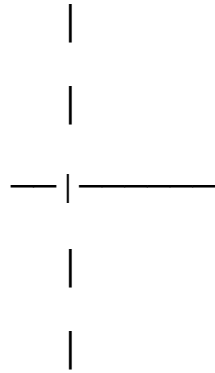
Similar points

All these points look almost identical, making reliable matching difficult.

Why Corner Detection is Better

A **corner** is a point where the intensity changes significantly in **two or more directions**. This makes corners much more distinctive than points along a straight edge.

For example:



Corner

A corner contains strong local information in multiple directions, so it can be used as a reliable **keypoint**.

Common corner detection methods include:

- **Harris Corner Detector**
- **Shi-Tomasi Corner Detector**
- **FAST**

Advantages of Corner Detection

1. **Distinctive keypoints:** Corners provide unique locations for matching.
2. **Better correspondence:** The same corner can be identified in different images.
3. **Useful for feature matching:** Corners are commonly used in image stitching, object recognition, and tracking.
4. **Works well in textured regions:** Textured areas contain many intensity changes in different directions, producing useful keypoints.
5. **Better localization:** A corner represents a specific point rather than an entire edge.

Edge vs Corner :

Edge Detection

Detects boundaries

Usually intensity changes in one direction

Produces edges/lines

Less suitable for point matching

Examples: Canny, Sobel

Corner Detection

Detects distinctive points

Intensity changes in multiple directions

Produces keypoints

Highly suitable for point matching

Examples: Harris, Shi-Tomasi, FAST

Example Application

Consider two photographs of the same building taken from slightly different viewpoints. Edge detection may detect the building's long horizontal and vertical edges, but there may be thousands of similar points along those edges.

Corner detection can identify distinctive locations such as:

- Window corners
- Door corners
- Building intersections
- Object boundaries
- Texture junctions

These points can then be matched between the two images.

Conclusion

Edge detection alone is insufficient for reliable image matching because **edges contain continuous and often repetitive information without providing unique point locations**. Corner detection identifies points where intensity changes in multiple directions, making them **distinctive, stable, and suitable as keypoints for matching**. Therefore, for an image containing straight edges and textured regions, combining edge information with **corner/keypoint detection** provides much better results for feature matching and object recognition.

3. Hough Transform in Noisy Images

A system uses Hough Transform to detect lines in noisy images but produces inaccurate results.

Analyze the effect of noise on detection and suggest improvements based on preprocessing or parameter selection.

Answer :

The **Hough Transform** is a feature extraction technique used to detect geometric shapes such as **straight lines and circles** in an image. When the input image contains noise, the Hough Transform may produce inaccurate or false detections because noise can be interpreted as additional edges.

Effect of Noise on Hough Transform

In a typical line-detection system, the process is:

Input Image → Preprocessing → Edge Detection → Hough Transform → Detected Lines

When noise is present, it affects the edge detection stage and consequently the Hough Transform.

1. **False edges:** Random noise creates unwanted intensity changes, which appear as edges.
2. **False lines:** These unwanted edges can accumulate votes in Hough space and produce lines that do not actually exist.

3. **Broken lines:** Noise can interrupt real edges, causing a genuine line to receive fewer votes.
4. **Incorrect line parameters:** Noise can shift the detected angle or distance of a line.
5. **Multiple detections:** The same actual line may be detected several times because of noisy or thick edges.

Improvements Using Preprocessing

1. Gaussian Blur

Applying **Gaussian Blur** before edge detection reduces high-frequency noise.

Noisy Image



Gaussian Blur



Canny Edge Detection



Hough Transform

This produces cleaner edges and reduces false line detections.

2. Median Filtering

For images containing **salt-and-pepper noise**, Median Filtering can be more effective because it removes isolated noisy pixels while preserving edges.

3. Proper Edge Detection

Using the **Canny edge detector** before Hough Transform helps obtain cleaner and thinner edges.

The Canny thresholds should be selected appropriately. Very low thresholds may detect noise, while very high thresholds may remove genuine edges.

Improvements Through Parameter Selection

The accuracy of Hough Transform also depends on its parameters.

1. Accumulator Threshold

The accumulator threshold determines how many votes are required for a line to be detected.

- **Low threshold:** More lines are detected, including false lines caused by noise.
- **High threshold:** Only strong lines are detected, reducing false positives.

Therefore, increasing the threshold can improve detection in noisy images, but an excessively high value may miss weak real lines.

2. Resolution of ρ and θ

The Hough Transform represents a line using:

$$P = x \cos\theta + y \sin \theta$$

The ρ and θ resolutions determine how precisely lines are represented.

- Very fine resolution can make the system more sensitive to noise.
- Very coarse resolution may merge nearby lines.

A suitable balance should be selected according to the image.

3. Minimum Line Length

When using **Probabilistic Hough Transform**, setting an appropriate minimum line length helps eliminate short lines caused by noise.

4. Maximum Line Gap

The maximum allowed gap between points belonging to the same line should be selected carefully.

- Too small → real lines may be broken.
- Too large → unrelated noisy points may be joined.

Recommended Approach

A practical solution is:

Original Noisy Image



Gaussian / Median Filtering



Grayscale Conversion



Canny Edge Detection



Hough Line Transform



Apply suitable threshold



Filter short/unwanted lines



Final Detected Lines

Conclusion

Noise can significantly reduce the accuracy of the Hough Transform by generating **false edges, false lines, and incorrect line parameters**. The problem can be reduced by applying preprocessing techniques such as **Gaussian or Median filtering**, followed by proper edge detection. In addition, selecting suitable **Hough threshold, ρ/θ resolution, minimum line length, and maximum line gap** parameters helps suppress false detections while preserving genuine lines. Therefore, combining effective preprocessing with appropriate parameter tuning produces more reliable line detection in noisy images.

4. Feature Matching under Transformations

Two images of the same object are taken at different scales and orientations. Traditional feature matching fails, but scale invariant feature matching works effectively. Analyze why scale invariant methods perform better in this scenario.

Answer :

When two images contain the **same object but are captured at different scales and orientations**, traditional feature matching may fail because the appearance and position of the features change. **Scale-invariant feature matching methods** perform better because they are designed to identify features that remain recognizable even when the image is resized or rotated.

Why Traditional Feature Matching Fails

Traditional methods may compare image pixels or simple local features directly.

For example, suppose an object appears like this in the first image:

Image 1





After scaling:

Image 2



The same object occupies a different number of pixels. Therefore, direct pixel-to-pixel comparison does not produce a good match.

Similarly, when the object is rotated, the orientation of edges and patterns changes. A feature that was originally horizontal may become diagonal or vertical.

Problems Caused by Transformations

1. Scale variation

When an object is closer to the camera, its features appear larger. When it is farther away, the features become smaller. A fixed-size feature detector may therefore fail to find corresponding points.

2. Rotation

Rotation changes the orientation of local image structures. A traditional feature descriptor that depends on a fixed orientation may describe the same feature differently in the second image.

3. Change in feature location

Scaling and rotation also change the coordinates of feature points, making direct coordinate-based matching unreliable.

Why Scale-Invariant Methods Perform Better

Scale-invariant feature methods detect and describe features based on their **local structure rather than their absolute size or orientation**.

Examples include:

- **SIFT (Scale-Invariant Feature Transform)**
- **SURF**
- **ORB** — provides rotation invariance and is designed to be efficient; its scale robustness is more limited than SIFT's.

1. Scale Invariance

Methods such as **SIFT** detect features at different scales using a scale-space representation.

Therefore, the same keypoint can potentially be detected whether it appears:

- Small
- Medium
- Large

This allows corresponding features to be found even when the object is resized.

2. Rotation Invariance

The feature is assigned a dominant orientation based on the local image gradient.

For example:

Original Rotated



/ \

Even though the feature's orientation changes, the descriptor can be constructed relative to its dominant orientation. This makes matching more reliable.

3. Local Feature Descriptors

Scale-invariant methods create descriptors that represent the **local appearance around a keypoint**.

Instead of comparing the entire image, they compare distinctive local regions.

For example:

Image 1

Image 2

[Feature A] —————→ [Feature A']

[Feature B] —————→ [Feature B']

[Feature C] —————→ [Feature C']

Even if the object has changed scale or orientation, the local structures can still be matched.

Comparison

Traditional Matching

Sensitive to scale

Sensitive to rotation

Often relies on direct appearance/pixel similarity

Matching may fail after transformations

Less suitable for varying viewpoints

Scale-Invariant Matching

Handles scale changes

Handles rotation

Uses local feature descriptors

More robust to transformations

Better for transformed images

Example Application

Suppose a mobile camera photographs a **car** from two different distances. In the first image, the car is small, while in the second image it is much larger. If the camera or car is rotated, the appearance changes further.

A scale-invariant method such as **SIFT** can detect distinctive features such as:

- Headlight corners
- Logo details
- Door edges
- Wheel structures

These features can be described and matched even when their scale and orientation differ.

Conclusion

Traditional feature matching performs poorly when images undergo **scaling and rotation** because the same features may have different sizes, orientations, and pixel arrangements. Scale-invariant methods such as **SIFT** overcome these limitations by detecting features across multiple scales and creating descriptors that are robust to orientation changes. Therefore, they provide **more reliable and accurate feature matching under scale and rotational transformations**, making them useful for object recognition, image stitching, tracking, and image retrieval.

5. PCA for Feature Optimization

A system extracts a large number of features, leading to high computational cost and redundancy.

Analyze how Principal Component Analysis helps in reducing dimensionality and improving efficiency.

Answer :

Principal Component Analysis (PCA) is a dimensionality reduction technique used to reduce a large number of features into a smaller set of meaningful features. In computer vision, extracting hundreds or thousands of features can increase computational cost and may contain redundant or highly correlated information. PCA helps solve this problem while retaining most of the important information.

Problems with a Large Number of Features

When a vision system extracts a large number of features:

1. **High computational cost** – More features require more processing during training and prediction.
2. **High memory usage** – Storing large feature vectors requires more memory.
3. **Redundant information** – Some features may contain similar or correlated information.
4. **Slower feature matching** – Comparing high-dimensional feature vectors takes more time.
5. **Curse of dimensionality** – Very high-dimensional data can make machine-learning algorithms less efficient and may reduce generalization.

How PCA Reduces Dimensionality

PCA transforms the original features into a new set of variables called **principal components**.

The principal components are arranged according to the amount of variance they contain:

- **First principal component (PC1):** Maximum variance
- **Second principal component (PC2):** Next highest variance
- **Third principal component (PC3):** Next highest variance
- And so on.

Therefore, instead of using all original features, we can retain only the first few components that contain most of the useful information.

Example

Suppose an image-processing system extracts **100 features**.

Original Features

F1 F2 F3 F4 ... F100

↓

PCA

↓

PC1 PC2 PC3 ... PC15

If the first **15 principal components preserve 95% of the variance**, the system can use only these 15 components instead of all 100 features.

This gives approximately:

100→15

features while retaining most of the important information.

Steps in PCA

1. Collect the feature data

A feature matrix is created from the extracted image features.

2. Standardize the features

Features are usually normalized so that features with larger numerical ranges do not dominate the analysis.

3. Calculate the covariance matrix

The covariance matrix identifies relationships and correlations between features.

4. Calculate eigenvectors and eigenvalues

- **Eigenvectors** represent the directions of the principal components.
- **Eigenvalues** indicate the amount of variance represented by each component.

5. Select principal components

Components are sorted according to their eigenvalues. The components containing the highest variance are selected.

6. Transform the data

The original high-dimensional feature vectors are projected onto the selected principal components.

Benefits in Computer Vision

PCA provides several advantages:

1. Reduced computation

Fewer features mean fewer calculations during classification, recognition, and matching.

2. Reduced memory requirements

Smaller feature vectors require less storage.

3. Removal of redundancy

Correlated features are combined into principal components, reducing duplicate information.

4. Faster processing

Feature matching and machine-learning algorithms can operate faster on lower-dimensional data.

5. Noise reduction

Low-variance components may contain noise. Removing them can improve the quality of the feature representation.

6. Better visualization

High-dimensional features can sometimes be reduced to **2 or 3 principal components** for visualization.

Example in Object Recognition

Consider a face-recognition system that extracts **500 features** from each face.

Using all 500 features:

500 Features



High computation



Slow recognition

After PCA:

500 Features



PCA



50 Principal Components



Faster recognition

If those 50 components preserve most of the important variation in the original data, the system can perform recognition much more efficiently.

Important Limitation

PCA maximizes **variance**, not necessarily classification accuracy. Therefore, a component with low variance may still be useful for distinguishing two classes. PCA should therefore be evaluated carefully before selecting the final number of components.

Conclusion

PCA helps optimize computer vision features by transforming a **high-dimensional feature set into a smaller set of uncorrelated principal components** while preserving most of the important information. By eliminating redundancy and low-information components, PCA reduces **computational cost, memory usage, and processing time**. Hence, PCA is an effective technique for improving the efficiency of feature-based computer vision and machine-learning systems.